

OSSP

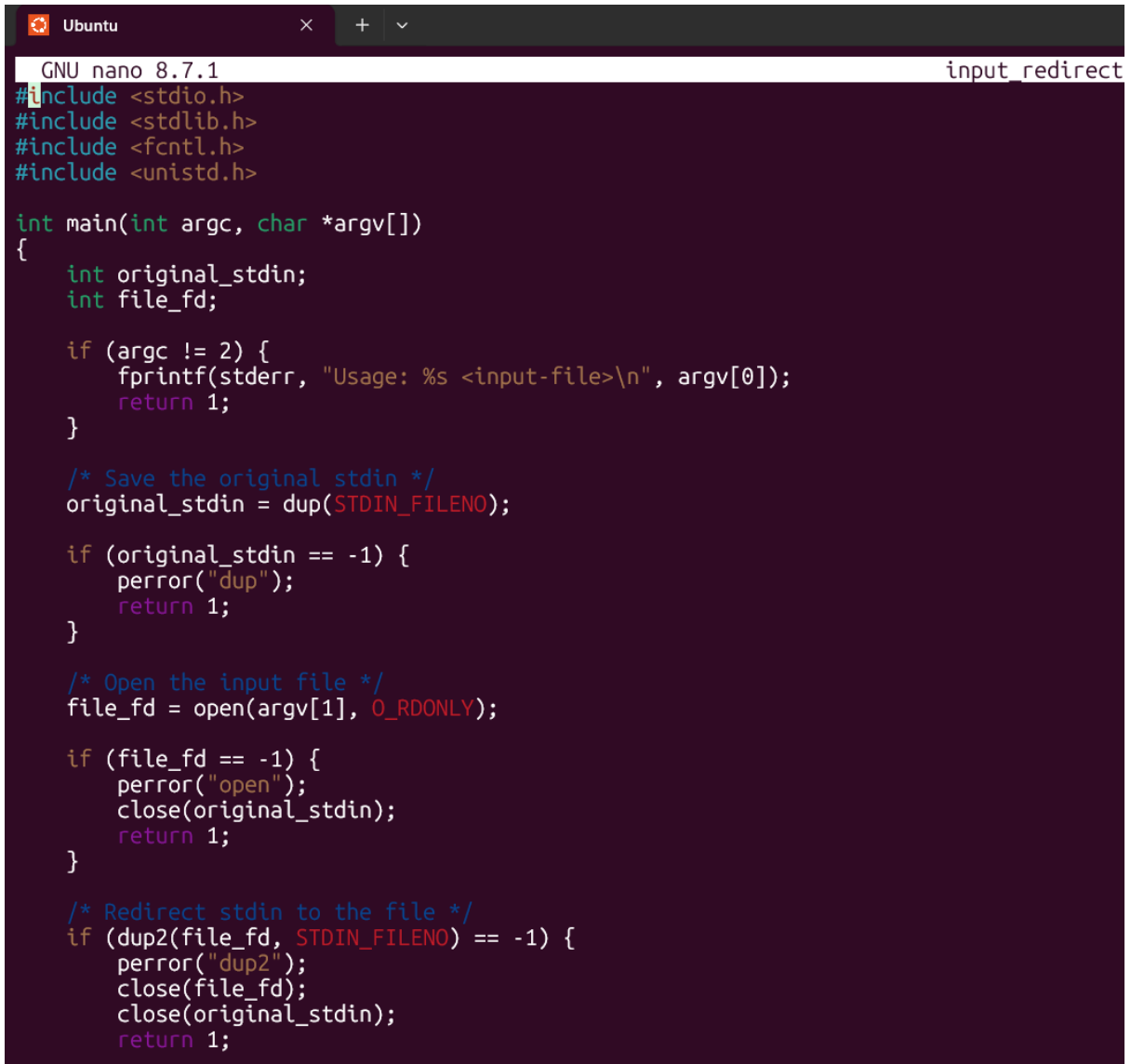
Skill Week -7

Roll No – 2520090085

Name - : Vakkalanka Sai Bhaskara Sujit

Task 1-:

To Parse Input Redirection, Open Files, Redirectstdin, Handle MissingFiles, Restore Streams,Test File Inputs



```
GNU nano 8.7.1 input_redirect
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>

int main(int argc, char *argv[])
{
    int original_stdin;
    int file_fd;

    if (argc != 2) {
        fprintf(stderr, "Usage: %s <input-file>\n", argv[0]);
        return 1;
    }

    /* Save the original stdin */
    original_stdin = dup(STDIN_FILENO);

    if (original_stdin == -1) {
        perror("dup");
        return 1;
    }

    /* Open the input file */
    file_fd = open(argv[1], O_RDONLY);

    if (file_fd == -1) {
        perror("open");
        close(original_stdin);
        return 1;
    }

    /* Redirect stdin to the file */
    if (dup2(file_fd, STDIN_FILENO) == -1) {
        perror("dup2");
        close(file_fd);
        close(original_stdin);
        return 1;
    }
}
```

```
Ubuntu × + ▾
GNU nano 8.7.1 in
    perror("dup2");
    close(file_fd);
    close(original_stdin);
    return 1;
}

close(file_fd);

printf("Reading from file: %s\n", argv[1]);

/* Read from redirected stdin */
char buffer[256];

while (fgets(buffer, sizeof(buffer), stdin) != NULL) {
    printf("Input: %s", buffer);
}

/* Restore original stdin */
if (dup2(original_stdin, STDIN_FILENO) == -1) {
    perror("restore stdin");
    close(original_stdin);
    return 1;
}

close(original_stdin);

printf("\nstdin restored.\n");

/* Test that stdin works again */
printf("Enter something: ");

if (fgets(buffer, sizeof(buffer), stdin) != NULL) {
    printf("You entered: %s", buffer);
}

return 0;
}
```

```
Ubuntu × + ▾
nocturnal@nocturnal:~$ mkdir -p ~/input_redirect
cd ~/input_redirect
nano input_redirect.c
nocturnal@nocturnal:~/input_redirect$ gcc -Wall -Wextra -o input_redirect input_redirect.c
nocturnal@nocturnal:~/input_redirect$ ls
input_redirect  input_redirect.c
nocturnal@nocturnal:~/input_redirect$ echo "Hello from the input file!" > input.txt
echo "This is the second line." >> input.txt
nocturnal@nocturnal:~/input_redirect$ cat input.txt
Hello from the input file!
This is the second line.
nocturnal@nocturnal:~/input_redirect$ ./input_redirect input.txt
Reading from file: input.txt
Input: Hello from the input file!
Input: This is the second line.

stdin restored.
Enter something: nocturnal@nocturnal:~/input_redirect$ Hello WSL
```

```
nocturnal@nocturnal:~/input_redirect$  
nocturnal@nocturnal:~/input_redirect$ ./input_redirect input.txt  
Reading from file: input.txt  
Input: Hello from the input file!  
Input: This is the second line.  
  
stdin restored.  
Enter something: nocturnal@nocturnal: nano input_redirect.c  
nocturnal@nocturnal:~/input_redirect$ ./input_redirect input.txt  
Reading from file: input.txt  
Input: Hello from the input file!  
Input: This is the second line.  
  
stdin restored.  
Enter something: nocturnal@nocturnal: gcc -Wall -Wextra -o input_redirect input_redirect.c input_redirect.c  
nocturnal@nocturnal:~/input_redirect$ echo "Hello from the input file!" > input.txt  
echo "This is the second line." >> input.txt  
nocturnal@nocturnal:~/input_redirect$ cat input.txt  
Hello from the input file!  
This is the second line.  
nocturnal@nocturnal:~/input_redirect$ ./input_redirect input.txt  
Reading from file: input.txt  
Input: Hello from the input file!  
Input: This is the second line.  
  
stdin restored.  
Enter something: Hello WSL  
You entered: Hello WSL  
nocturnal@nocturnal:~/input_redirect$ ./input_redirect missing.txt  
open: No such file or directory  
nocturnal@nocturnal:~/input_redirect$
```

Task-2:

To Parse OutputRedirection, Create OutputFiles, Redirect stdout,Handle Permissions, Verify Output Data, Test Redirection

A screenshot of a terminal window with a dark background. The title bar at the top shows 'Ubuntu' with a red logo, and window controls (close, maximize, and a dropdown menu). The terminal content shows the GNU nano 8.7.1 editor with a C program. The program includes standard headers, defines a BUFFER_SIZE of 256, and implements a show_error function. The main function checks for correct argument usage (argc == 3), verifies the presence of an output redirection operator (>), and checks if an output filename was provided. The code is color-coded: keywords in green, comments in blue, and strings in yellow. The word 'redire' is visible at the top right of the editor area.

```
GNU nano 8.7.1 redire
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <errno.h>

#define BUFFER_SIZE 256

void show_error(const char *message)
{
    fprintf(stderr, "Error: %s: %s\n", message, strerror(errno));
}

int main(int argc, char *argv[])
{
    int saved_stdout;
    int output_fd;
    char message[BUFFER_SIZE];

    /* Check arguments */
    if (argc != 3)
    {
        fprintf(stderr, "Usage: %s > output_file\n", argv[0]);
        fprintf(stderr, "Example: %s \">\" result.txt\n", argv[0]);
        return EXIT_FAILURE;
    }

    /* Check for output redirection operator */
    if (strcmp(argv[1], ">") != 0)
    {
        fprintf(stderr, "Error: expected '>' before output filename.\n");
        return EXIT_FAILURE;
    }

    /* Check output filename */
    if (strlen(argv[2]) == 0)
    {

```

```
Ubuntu × + ▾
GNU nano 8.7.1 redirect_output.c *
/* Check output filename */
if (strlen(argv[2]) == 0)
{
    fprintf(stderr, "Error: output filename is empty.\n");
    return EXIT_FAILURE;
}

/*
 * Save the current stdout so that
 * it can be restored later.
 */
saved_stdout = dup(STDOUT_FILENO);

if (saved_stdout == -1)
{
    show_error("cannot save stdout");
    return EXIT_FAILURE;
}

/*
 * Create the output file.
 *
 * O_WRONLY -> write only
 * O_CREAT  -> create if it does not exist
 * O_TRUNC  -> erase old contents
 * 0644     -> file permissions
 */
output_fd = open(
    argv[2],
    O_WRONLY | O_CREAT | O_TRUNC,
    0644
);

if (output_fd == -1)
{
    show_error("cannot create output file");
    close(saved_stdout);
    return EXIT_FAILURE;
}
```

```
Ubuntu × + ▾
GNU nano 8.7.1 redirect
}

close(output_fd);

/*
 * Everything printed using printf()
 * now goes into the output file.
 */
printf("Output redirection is working.\n");
printf("This text was written using stdout.\n");
printf("Output file: %s\n", argv[2]);

snprintf(
    message,
    sizeof(message),
    "The file was successfully created and written.\n"
);

printf("%s", message);

/* Make sure all output is written */
fflush(stdout);

/* Restore stdout */
if (dup2(saved_stdout, STDOUT_FILENO) == -1)
{
    show_error("cannot restore stdout");
    close(saved_stdout);
    return EXIT_FAILURE;
}

close(saved_stdout);

printf("stdout restored successfully.\n");
printf("Output was redirected to: %s\n", argv[2]);

return EXIT_SUCCESS;
}
```

```
nocturnal@nocturnal:~/input_redirect$ ./redirect_output ">" result.txt
stdout restored successfully.
Output was redirected to: result.txt
nocturnal@nocturnal:~/input_redirect$ ls -l result.txt
-rw-r--r-- 1 nocturnal nocturnal 138 Sep 24 09:41 result.txt
nocturnal@nocturnal:~/input_redirect$ cat result.txt
Output redirection is working.
This text was written using stdout.
Output file: result.txt
The file was successfully created and written.
nocturnal@nocturnal:~/input_redirect$ ./redirect_output "<" test.txt
Error: expected '>' before output filename.
nocturnal@nocturnal:~/input_redirect$ ./redirect_output ">" /root/test.txt
Error: cannot create output file: Permission denied
nocturnal@nocturnal:~/input_redirect$ ./redirect_output ">" result.txt
cat result.txt
stdout restored successfully.
Output was redirected to: result.txt
Output redirection is working.
This text was written using stdout.
Output file: result.txt
The file was successfully created and written.
nocturnal@nocturnal:~/input_redirect$ █
```

Task 3 -:

To Parse Append Operator, Open Files in Append Mode, Redirect Streams, Preserve Existing Data, Validate Appends, Test Scenarios

```
Ubuntu × + ▾
GNU nano 8.7.1 append_redirect.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <errno.h>

#define BUFFER_SIZE 256

void show_error(const char *message)
{
    fprintf(stderr, "Error: %s: %s\n", message, strerror(errno));
}

int main(int argc, char *argv[])
{
    int saved_stdout;
    int output_fd;
    char buffer[BUFFER_SIZE];

    /* Check command-line arguments */
    if (argc != 3)
    {
        fprintf(stderr, "Usage: %s \">>>\" output_file\n", argv[0]);
        fprintf(stderr, "Example: %s \">>>\" log.txt\n", argv[0]);
        return EXIT_FAILURE;
    }

    /* Check for append operator */
    if (strcmp(argv[1], ">>") != 0)
    {
        fprintf(stderr, "Error: expected '>>' append operator.\n");
        return EXIT_FAILURE;
    }

    /* Check output filename */
    if (strlen(argv[2]) == 0)
    {

```

```
GNU nano 8.7.1 append_redirect.c
if (strlen(argv[2]) == 0)
{
    fprintf(stderr, "Error: output filename cannot be empty.\n");
    return EXIT_FAILURE;
}

/* Save original stdout */
saved_stdout = dup(STDOUT_FILENO);

if (saved_stdout == -1)
{
    show_error("cannot save stdout");
    return EXIT_FAILURE;
}

/*
 * Open file in append mode.
 * O_WRONLY -> write only
 * O_CREAT  -> create if it doesn't exist
 * O_APPEND -> write at the end
 */
output_fd = open(
    argv[2],
    O_WRONLY | O_CREAT | O_APPEND,
    0644
);

if (output_fd == -1)
{
    show_error("cannot open file for appending");
    close(saved_stdout);
    return EXIT_FAILURE;
}

/* Redirect stdout to the output file */
if (dup2(output_fd, STDOUT_FILENO) == -1)
{
    show_error("cannot redirect stdout");
}
```

```
        close(output_fd);
        close(saved_stdout);
        return EXIT_FAILURE;
    }

    close(output_fd);

    /* Write data to the file */
    printf("New appended line.\n");
    printf("This data was added using append redirection.\n");

    snprintf(
        buffer,
        sizeof(buffer),
        "Append operation completed successfully.\n"
    );

    printf("%s", buffer);

    /* Flush stdout before restoring */
    fflush(stdout);

    /* Restore original stdout */
    if (dup2(saved_stdout, STDOUT_FILENO) == -1)
    {
        show_error("cannot restore stdout");
        close(saved_stdout);
        return EXIT_FAILURE;
    }

    close(saved_stdout);

    printf("stdout restored successfully.\n");
    printf("Data appended to: %s\n", argv[2]);

    return EXIT_SUCCESS;
}
```

```
nocturnal@nocturnal:~/append_redirection$ nano append_redirect.c
nocturnal@nocturnal:~/append_redirection$ ls
append_redirect.c
nocturnal@nocturnal:~/append_redirection$ gcc -Wall -Wextra -o append_redirect append_redirect.c
nocturnal@nocturnal:~/append_redirection$ ls
append_redirect  append_redirect.c
nocturnal@nocturnal:~/append_redirection$ echo "Original line 1" > append_test.txt
nocturnal@nocturnal:~/append_redirection$ echo "Original line 2" >> append_test.txt
nocturnal@nocturnal:~/append_redirection$ cat append_test.txt
Original line 1
Original line 2
nocturnal@nocturnal:~/append_redirection$ ./append_redirect ">>" append_test.txt
stdout restored successfully.
Data appended to: append_test.txt
nocturnal@nocturnal:~/append_redirection$ cat append_test.txt
Original line 1
Original line 2
New appended line.
This data was added using append redirection.
Append operation completed successfully.
nocturnal@nocturnal:~/append_redirection$ ./append_redirect ">>" append_test.txt
stdout restored successfully.
Data appended to: append_test.txt
nocturnal@nocturnal:~/append_redirection$ cat append_test.txt
Original line 1
Original line 2
New appended line.
This data was added using append redirection.
Append operation completed successfully.
nocturnal@nocturnal:~/append_redirection$ ./append_redirect ">" append_test.txt
Error: expected '>>' append operator.
nocturnal@nocturnal:~/append_redirection$ ./append_redirect ">>" /root/test_append.txt
Error: cannot open file for appending: Permission denied
nocturnal@nocturnal:~/append_redirection$ ./append_redirect ">>" new_append.txt
stdout restored successfully.
Data appended to: new_append.txt
nocturnal@nocturnal:~/append_redirection$ V
V: command not found
nocturnal@nocturnal:~/append_redirection$ cat new_append.txt
New appended line.
This data was added using append redirection.
Append operation completed successfully.
nocturnal@nocturnal:~/append_redirection$
```

Task 4-:

To Parse stderr Redirection, Redirect Error Streams, Capture Error Output, Verify File Contents, Handle Failures, Test Error Cases

```
Ubuntu × + ▾
GNU nano 8.7.1 error_redirec
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <errno.h>

void show_error(const char *message)
{
    fprintf(stderr, "Error: %s: %s\n", message, strerror(errno));
}

int main(int argc, char *argv[])
{
    int saved_stderr;
    int error_fd;

    /* Check command-line arguments */
    if (argc != 3)
    {
        fprintf(stderr, "Usage: %s \"2>\" error_file\n", argv[0]);
        fprintf(stderr, "Example: %s \"2>\" errors.txt\n", argv[0]);
        return EXIT_FAILURE;
    }

    /* Check stderr redirection operator */
    if (strcmp(argv[1], "2>") != 0)
    {
        fprintf(stderr, "Error: expected '2>' stderr redirection operator.\n");
        return EXIT_FAILURE;
    }

    /* Save the original stderr */
    saved_stderr = dup(STDERR_FILENO);

    if (saved_stderr == -1)
    {
        perror("dup");
    }
}
```

```
Ubuntu × + ▾
GNU nano 8.7.1 error_redirect.c *
*/
error_fd = open(
    argv[2],
    O_WRONLY | O_CREAT | O_TRUNC,
    0644
);

if (error_fd == -1)
{
    dprintf(saved_stderr,
        "Error: cannot open error file: %s\n",
        strerror(errno));

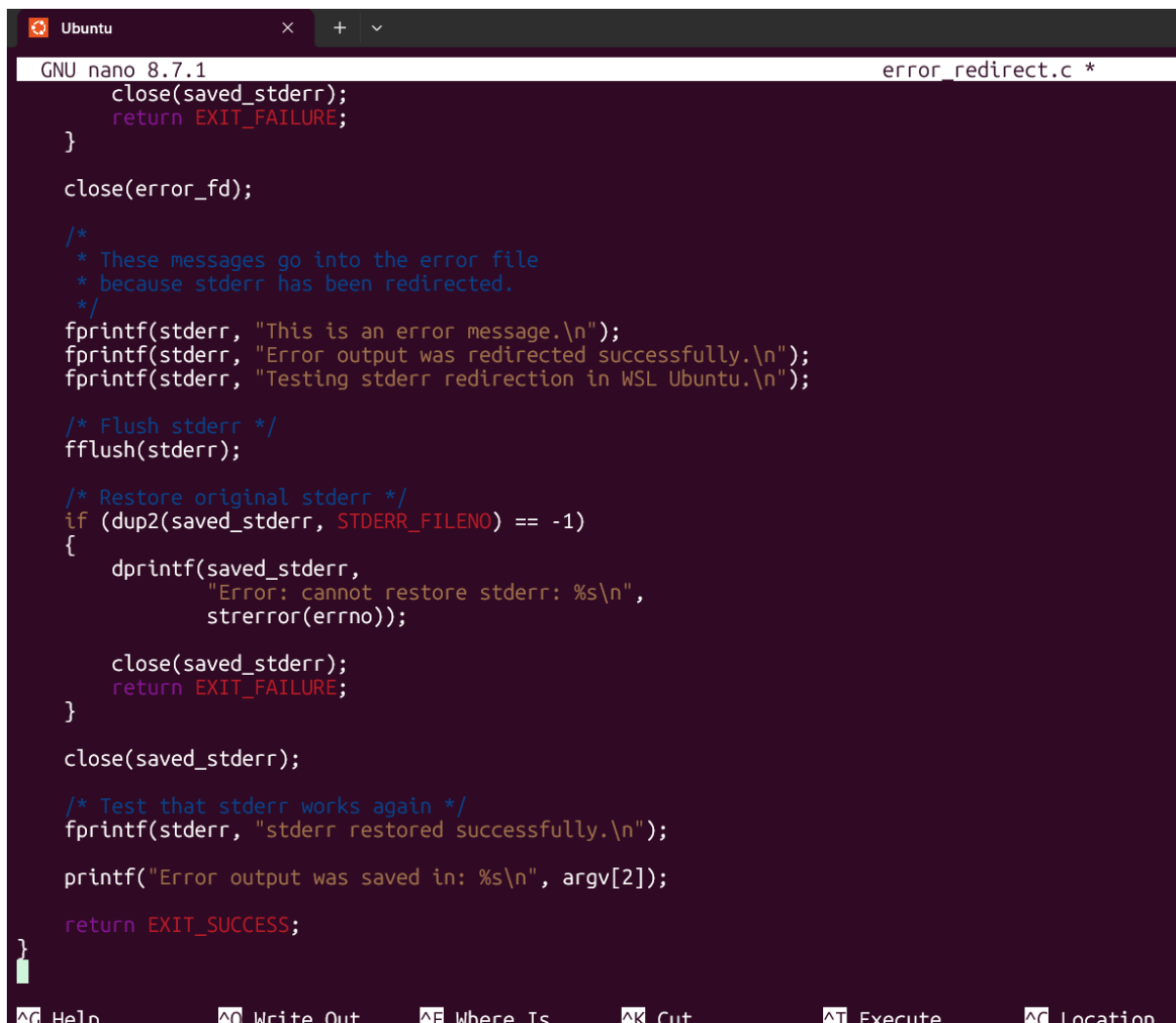
    close(saved_stderr);
    return EXIT_FAILURE;
}

/* Redirect stderr to the error file */
if (dup2(error_fd, STDERR_FILENO) == -1)
{
    dprintf(saved_stderr,
        "Error: cannot redirect stderr: %s\n",
        strerror(errno));

    close(error_fd);
    close(saved_stderr);
    return EXIT_FAILURE;
}

close(error_fd);

/*
 * These messages go into the error file
 * because stderr has been redirected.
 */
fprintf(stderr, "This is an error message.\n");
fprintf(stderr, "Error output was redirected successfully.\n");
fprintf(stderr, "Testing stderr redirection in WSL Ubuntu.\n");
```



```
GNU nano 8.7.1 error_redirect.c *
    close(saved_stderr);
    return EXIT_FAILURE;
}

close(error_fd);

/*
 * These messages go into the error file
 * because stderr has been redirected.
 */
fprintf(stderr, "This is an error message.\n");
fprintf(stderr, "Error output was redirected successfully.\n");
fprintf(stderr, "Testing stderr redirection in WSL Ubuntu.\n");

/* Flush stderr */
fflush(stderr);

/* Restore original stderr */
if (dup2(saved_stderr, STDERR_FILENO) == -1)
{
    dprintf(saved_stderr,
            "Error: cannot restore stderr: %s\n",
            strerror(errno));

    close(saved_stderr);
    return EXIT_FAILURE;
}

close(saved_stderr);

/* Test that stderr works again */
fprintf(stderr, "stderr restored successfully.\n");

printf("Error output was saved in: %s\n", argv[2]);

return EXIT_SUCCESS;
}
```

Output-:

```
nocturnal@nocturnal:~/stderr_redirection$ nano error_redirect.c
nocturnal@nocturnal:~/stderr_redirection$ gcc -Wall -Wextra -o error_redirect error_redirect.c
nocturnal@nocturnal:~/stderr_redirection$ ls
error_redirect  error_redirect.c
nocturnal@nocturnal:~/stderr_redirection$ ./error_redirect "2>" errors.txt
stderr restored successfully.
Error output was saved in: errors.txt
nocturnal@nocturnal:~/stderr_redirection$ ./error_redirect "2>" errors.txt
stderr restored successfully.
Error output was saved in: errors.txt
nocturnal@nocturnal:~/stderr_redirection$ ls -l errors.txt
-rw-r--r-- 1 nocturnal nocturnal 110 Sep 24 10:01 errors.txt
nocturnal@nocturnal:~/stderr_redirection$ ./error_redirect "2>" errors.txt
stderr restored successfully.
Error output was saved in: errors.txt
nocturnal@nocturnal:~/stderr_redirection$ cat errors.txt
This is an error message.
Error output was redirected successfully.
Testing stderr redirection in WSL Ubuntu.
nocturnal@nocturnal:~/stderr_redirection$ O_TRUNC
O_TRUNC: command not found
nocturnal@nocturnal:~/stderr_redirection$ ./error_redirect ">" errors.txt
Error: expected '2>' stderr redirection operator.
nocturnal@nocturnal:~/stderr_redirection$ ^C
nocturnal@nocturnal:~/stderr_redirection$ ./error_redirect "2>" /root/errors.txt
Error: cannot open error file: Permission denied
nocturnal@nocturnal:~/stderr_redirection$ ./error_redirect
Usage: ./error_redirect "2>" error_file
Example: ./error_redirect "2>" errors.txt
nocturnal@nocturnal:~/stderr_redirection$ ./error_redirect "2>" test_errors.txt
stderr restored successfully.
Error output was saved in: test_errors.txt
nocturnal@nocturnal:~/stderr_redirection$
```